

mcp-kicad-vscode: automação do fluxo de projeto de placas de circuito impresso no KiCad por meio do Model Context Protocol

William da Silva Vianna
Instituto Federal Fluminense (IFF)
Campos dos Goytacazes, RJ, Brasil

Resumo

A crescente adoção de assistentes baseados em grandes modelos de linguagem (LLMs) na engenharia cria uma demanda por integrações padronizadas entre esses assistentes e ferramentas de automação de projeto eletrônico (EDA). Este trabalho analisa e avalia o `mcp-kicad-vscode`, um servidor *Model Context Protocol* (MCP) de código aberto (projeto `KiCAD-MCP-Server`, de *mixelpixx*) que expõe operações do KiCad — projeto, esquemático, layout de placa, roteamento, verificação de regras, exportação de fabricação e integrações com catálogos de componentes — a assistentes de IA. O sistema adota arquitetura em duas camadas (TypeScript para o protocolo MCP e Python para a interface com o *pcbnew*/IPC), registra 233 ferramentas organizadas em 24 módulos funcionais, disponibiliza 173 delas para descoberta por palavra-chave em 16 categorias e expõe 23 recursos dinâmicos de estado do projeto. A validação combinou métricas de implementação, execução das suítes de teste (84 casos TypeScript e 2.367 casos Python) e um estudo de caso, conduzido pelo autor, com uma placa seguidora de linhas de quatro camadas (97 *footprints*, 83 redes, 82,1 mm × 80,2 mm). Os resultados locais apontam 84/84 testes TypeScript aprovados e 2.328/2.367 casos Python aprovados (8 falhas e 31 ignorados), além de verificação de regras de projeto que identificou 11 violações residuais (9 erros; 2 avisos) e zero itens não conectados. Discutem-se limitações — integração IPC não exercitada no ambiente de validação, 60 ferramentas ainda não indexadas e ausência de estudo com usuários — e trabalhos futuros.

Palavras-chave: Model Context Protocol; KiCad; projeto de placas de circuito impresso; assistentes de IA; automação de EDA.

1 Introdução

O projeto de placas de circuito impresso (PCB, do inglês *printed circuit board*) é uma atividade multidisciplinar que combina decisões elétricas, térmicas, mecânicas e de manufatura. Nas últimas décadas, o setor consolidou fluxos de trabalho fortemente assistidos por computador, nos quais ferramentas de automação de projeto eletrônico (EDA) executam desde a captura esquemática até a geração de arquivos de fabricação, passando por verificação de regras de projeto (DRC, *design rule check*) e roteamento assistido [7, 5].

Paralelamente, assistentes de programação e engenharia baseados em grandes modelos de linguagem (LLMs) passaram a atuar como agentes capazes de planejar e executar ações em computadores, desde que recebam interfaces adequadas às suas capacidades [23, 22]. Quando o alvo é um *software* de engenharia desktop, como o KiCad, o obstáculo deixa de ser a capacidade de raciocínio do modelo e passa a ser a ausência de uma camada de integração padronizada, descobrível e verificável entre o assistente e as operações da ferramenta.

Em novembro de 2024, a Anthropic publicou o *Model Context Protocol* (MCP), um protocolo aberto baseado em JSON-RPC 2.0 que padroniza a forma como aplicações hospedeiras (*hosts*) expõem *tools*, *resources* e *prompts* a modelos de linguagem [1, 14]. A proposta reduz o custo

de integração de $N \times M$ conectores proprietários para um único protocolo, permitindo que um mesmo servidor seja utilizado por diferentes clientes (editores de código, ambientes de chat e agentes autônomos).

Este trabalho parte da seguinte questão de pesquisa: *como expor de forma ampla, descobrível e testável o fluxo de projeto de PCBs do KiCad a assistentes de IA, preservando a robustez exigida por operações destrutivas sobre arquivos de projeto?* Para respondê-la, este trabalho analisa e avalia o `mcp-kicad-vscode` — o servidor MCP de código aberto do projeto `KiCAD-MCP-Server`, criado por *mixelpixx* [12] —, que traduz operações do KiCad em ferramentas executáveis por agentes, com integração a catálogos reais de componentes e a roteador automático, e conduz um estudo de caso em que o servidor é aplicado ao projeto de uma placa seguidora de linhas de quatro camadas.

As principais contribuições deste trabalho são:

1. a análise documentada da arquitetura em duas camadas do servidor (TypeScript para o protocolo MCP; Python para a interface com a API `pcbnew`/IPC do KiCad), com abstração de *backends* e correlação de requisições entre os processos;
2. o mapeamento do catálogo de 233 ferramentas MCP organizadas em 24 módulos funcionais, cobrindo esquemático, placa, roteamento, regras de projeto, exportação, bibliotecas, criação de partes, datasheets e cadeia de suprimentos;
3. a avaliação dos mecanismos de descoberta por palavra-chave (`search_tools`, `get_category_tools`), que indexam 173 ferramentas em 16 categorias, e dos 23 recursos dinâmicos que expõem o estado do projeto;
4. a execução e a análise da estratégia de qualidade do projeto, que combina testes automatizados em duas linguagens (84 casos em TypeScript e 2.367 casos coletados em Python), integração contínua em múltiplos sistemas operacionais e geração automática do inventário de ferramentas;
5. uma avaliação por estudo de caso, conduzida pelo autor, com uma placa seguidora de linhas de quatro camadas projetada com o auxílio do servidor, incluindo verificação de regras por `kicad-cli`, quantificação de violações residuais e análise crítica das limitações.

O restante do artigo está organizado da seguinte forma: a Seção 2 apresenta a fundamentação teórica e os trabalhos relacionados; a Seção 3 descreve os materiais e métodos, incluindo a arquitetura do servidor e o desenho da avaliação; a Seção 4 reporta os resultados e a Seção 5 os discute; por fim, a Seção 6 conclui o trabalho e aponta direções futuras.

2 Fundamentação teórica e trabalhos relacionados

2.1 O Model Context Protocol

O MCP é um protocolo aberto que padroniza a comunicação entre aplicações de IA e fontes externas de contexto e execução [1]. Sua especificação define uma arquitetura cliente–servidor na qual o *host* (por exemplo, um editor de código ou assistente conversacional) instancia um cliente MCP por servidor conectado. A comunicação emprega JSON-RPC 2.0 e pode ocorrer por entrada/saída padrão (*stdio*) ou por transportes HTTP com streaming [14].

O protocolo define três primitivas principais que um servidor pode oferecer [13]:

- **Tools:** funções executáveis, descritas por esquema JSON, que o modelo pode invocar para produzir efeitos (criar arquivos, modificar geometria, executar verificações);
- **Resources:** dados somente leitura identificados por URI, como o estado de um projeto ou listas de componentes;

- **Prompts:** modelos de interação predefinidos que orientam o uso do servidor para tarefas recorrentes.

A especificação utilizada nesta implementação é a de 18 de junho de 2025 [14], que consolida o ciclo de vida de inicialização, negociação de capacidades e transporte. A adoção do MCP por diferentes fornecedores de modelos e editores reduziu o acoplamento entre assistentes e ferramentas, tornando viável publicar um único servidor para todo um domínio, como o projeto de PCBs.

2.2 Agentes baseados em LLMs e uso de ferramentas

A capacidade de um agente de interagir com o ambiente depende não apenas do modelo, mas do desenho da interface que lhe é oferecida. Yao et al. [23] demonstram que intercalar raciocínio e ação melhora o desempenho em tarefas que exigem decisões encadeadas. Schick et al. [19] e Patil et al. [16] evidenciam que modelos podem aprender a invocar APIs e ferramentas externas com acurácia crescente quando as chamadas são expostas de forma estruturada. Yang et al. [22] mostram, no contexto de engenharia de software, que a interface agente-computador (ACI) tem impacto direto no desempenho: nomes de comandos claros, saídas concisas e descobribilidade de ferramentas importam tanto quanto o modelo utilizado.

Esses resultados motivam decisões de projeto adotadas no servidor estudado: exposição individual de cada ferramenta (sem roteador oculto), descrições objetivas, mecanismos explícitos de descoberta e respostas padronizadas e testáveis.

2.3 Automação de EDA e o KiCad

O KiCad é uma suíte de EDA de código aberto que abrange captura esquemática, layout de placa, bibliotecas de símbolos e *footprints*, visualização 3D e geração de arquivos de fabricação [7]. Seus arquivos de projeto são expressões-S (*S-expressions*) versionáveis em texto, o que facilita automação e revisão por ferramentas externas.

A extensibilidade do KiCad se apoia em três mecanismos complementares: (i) a API Python `pcbnew`, disponibilizada por meio de *bindings* SWIG [20]; (ii) a API IPC, que permite comunicação em tempo real com a interface gráfica; e (iii) o utilitário de linha de comando `kicad-cli`, que executa exportações e verificações de forma independente da interface gráfica [9, 8]. Esses mecanismos são a base de integrações automatizadas, inclusive as conduzidas por agentes de IA.

2.4 Trabalhos relacionados

A aplicação de LLMs ao domínio de *design* de circuitos tem exemplos relevantes. Liu et al. [11] adaptam modelos ao domínio de projeto de *chips* industriais (assistente de engenharia, geração de *scripts* de EDA e análise de defeitos), demonstrando ganhos com especialização de domínio. Trabalhos dessa natureza, no entanto, concentram-se na adaptação do *modelo* a um domínio; a integração entre assistentes genéricos e ferramentas de EDA permanece fragmentada, tipicamente resolvida por *scripts* específicos e não reutilizáveis.

Do lado das ferramentas, a automação do KiCad por *scripts* Python é consolidada, porém cada integração reproduz decisões de sessão, tratamento de erros e descoberta de operações. O MCP altera esse cenário ao oferecer um contrato único entre assistentes e ferramentas [14]. Neste contexto, o `mcp-kicad-vscode` — o servidor MCP de código aberto do projeto `KiCAD-MCP-Server`, criado por *mixelpixx* [12] — posiciona-se como uma solução de propósito geral para o fluxo de PCB no KiCad, com cobertura ampla, integrações de fabricação e suíte de testes abrangente — combinação ainda escassa na literatura e no ecossistema de código aberto.

A lacuna endereçada é, portanto, dupla: (i) ausência de uma camada padronizada e descobrível que cubra o ciclo de projeto de PCB de ponta a ponta (esquemático, layout, verificação,

fabricação); e (ii) ausência de evidências publicadas sobre robustez e manutenibilidade desse tipo de integração, incluindo o comportamento sob operações destrutivas e a cobertura de testes.

3 Materiais e métodos

3.1 Delineamento da pesquisa

Este trabalho caracteriza-se como pesquisa aplicada de natureza tecnológica, conduzida por meio da análise de um artefato de *software* existente e de avaliação por estudo de caso instrumental. O percurso metodológico compreendeu cinco etapas: (i) auditoria do domínio e do código-fonte do servidor; (ii) definição de requisitos de avaliação do sistema e da API MCP exposta; (iii) análise da arquitetura em duas camadas e do protocolo interno; (iv) verificação automatizada por suítes de teste e integração contínua; (v) estudo de caso com uma placa de quatro camadas projetada pelo autor, com o auxílio do servidor, no mesmo *workspace*.

Não se trata de experimento controlado com grupo de comparação: a avaliação adota métricas descritivas de implementação (contagem de ferramentas, recursos, linhas de código e casos de teste), resultados de execução das suítes e indicadores de verificação de projeto (regras do KiCad) obtidos sobre um artefato real. As ameaças à validade dessa escolha são discutidas na Seção 5.

3.2 Requisitos do sistema

Os requisitos que orientaram a avaliação estão sintetizados na Tabela 1, com a forma de atendimento observada e a evidência correspondente.

Tabela 1: Requisitos do servidor e evidências de atendimento.

ID	Requisito	Atendimento	Evidência
R1	Cobrir o fluxo de PCB de ponta a ponta	233 ferramentas em 24 módulos (projeto, esquemático, roteamento, regras, exportação)	Inventário gerado (<code>docs/TOOL_INVENTORY.md</code>) e Seção 4
R2	Permitir descoberta de operações pelo agente	Registro com categorias, lista de essenciais e <code>search_tools</code> / <code>get_category_tools</code>	173 ferramentas indexadas em 16 categorias
R3	Ser robusto sob falhas e respostas fora de ordem	Correlação de requisições com identificadores, descarte de respostas tardias, tempos-limite por comando	Testes de correlação e de <i>startup</i> em <code>tests-ts/</code>
R4	Preservar integridade dos arquivos do usuário	<i>Saves</i> explícitos, sessão fixada por <i>backend</i> , validação prévia de esquemáticos corrompidos	Suíte Python (<code>tests/</code>) e histórico de correções do projeto
R5	Ser portátil (Linux, Windows, macOS)	Node.js + Python portáveis; <i>launcher</i> com variáveis específicas por plataforma	Matriz de CI em quatro sistemas operacionais
R6	Ser verificável e extensível	Testes em duas linguagens, inventário gerado automaticamente como <i>gate</i> de CI	84 casos TypeScript e 2.367 casos Python; verificação <code>docs:tools:check</code>

3.3 Arquitetura do servidor

O sistema adota arquitetura em duas camadas, ilustrada na Figura 1. A camada externa implementa o protocolo MCP em TypeScript, registra as ferramentas com esquemas de validação e gerencia o ciclo de vida do processo Python. A camada interna, em Python, traduz comandos em operações do KiCad por meio de dois *backends* intercambiáveis: a API `pcbnew` (SWIG) e a

API IPC [9, 8, 20]. A seleção do *backend* é automática, com queda para SWIG quando a API IPC não está disponível.

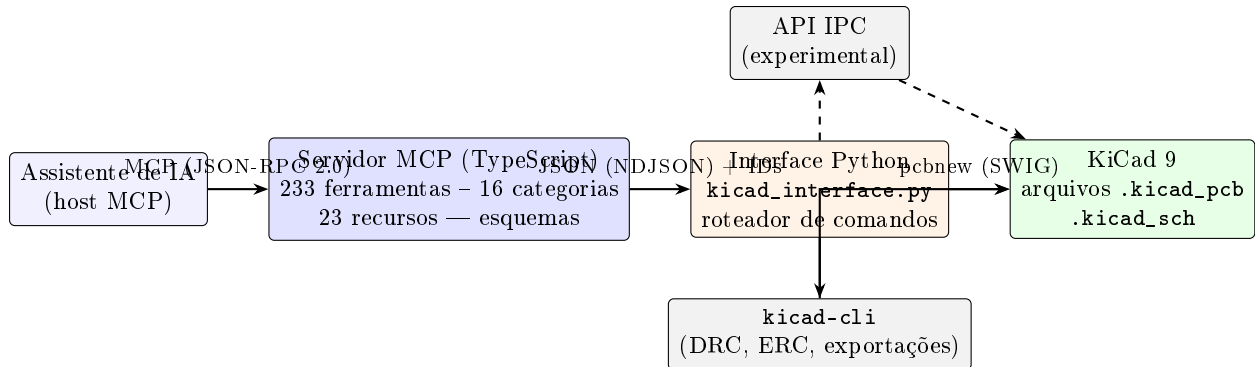


Figura 1: Arquitetura em duas camadas do `mcp-kicad-vscode`. A camada TypeScript fala MCP com o assistente; a camada Python opera o KiCad por SWIG, IPC (experimental) ou `kicad-cli`.

Cada ferramenta é registrada individualmente com `server.tool()`, isto é, sem roteador que oculte esquemas do cliente MCP. O registro de categorias existe para apoiar a *descoberta* — decisão alinhada à literatura sobre interfaces agente-computador [22]: o agente pode tanto chamar uma ferramenta conhecida quanto localizá-la por palavra-chave.

O protocolo interno entre as camadas usa JSON delimitado por linhas (*newline-delimited JSON*) com identificadores de requisição. Cada resposta ecoa o identificador correspondente; respostas tardias de uma requisição expirada são descartadas, evitando o desalinhamento entre pedido e resposta que pode ocorrer após um tempo-limite. O processo Python é único e persistente, iniciado na primeira chamada de ferramenta, e mantém o estado do projeto em memória quando o *backend* permite.

3.4 Estratégia de qualidade

A estratégia de qualidade do servidor adota três mecanismos complementares: (i) suítes de teste em TypeScript (Vitest [21]) e Python (pytest [17]), incluindo testes de correlação de requisições, completude do registro de ferramentas e validação sintática; (ii) integração contínua [4] com matriz de sistemas operacionais e versões de *runtime*; e (iii) geração automática do inventário de ferramentas a partir do código, exigida como condição de aceitação no CI (`npm run docs:tools:check`), impedindo que a documentação diverga da implementação.

3.5 Ambiente de validação

A execução local ocorreu em estação de trabalho com Ubuntu 24.04.4 LTS (*kernel* 7.0.0-31), com KiCad 9.0.7 (pacote *snap*, revisão 22), Node.js v22.23.2 [15], npm 12.0.2, Python 3.12.3 [18] (ambiente virtual com as dependências de produção e de teste declaradas pelo projeto) e pytest 9.1.1. O utilitário `kicad-cli` 9.0.7 foi utilizado para verificação e exportações fora da interface gráfica [9].

3.6 Estudo de caso

O estudo de caso utiliza a placa `seguidor_de_linhas_4camada2-2026-1`, um projeto de quatro camadas para um robô seguidor de linhas, projetado pelo autor com o auxílio do servidor, contendo subsistemas de sensoramento óptico reflexivo, conversão de energia (*MPPT*), acionamento de motores em ponte H, interface USB-C, regulação de tensão e sinalização. O projeto foi analisado por meio de: (i) contagem de *footprints*, redes e folhas do esquemático; (ii) execução de DRC com `kicad-cli pcb drc -format json -severity-all`; (iii) renderizações 3D das faces

superior e inferior; e (iv) inspeção dos artefatos de fabricação gerados pelo fluxo de projeto. Os comandos exatos e os artefatos resultantes estão preservados no repositório do projeto (material suplementar digital), em especial no diretório **evidencias/**.

4 Resultados

4.1 Métricas de implementação e catálogo de ferramentas

A implementação reúne 42 arquivos TypeScript (14.677 linhas) na camada MCP e 54.442 linhas em Python na camada de interface, das quais 38.050 no módulo de comandos e 6.529 no roteador principal (`kicad_interface.py`). O servidor registra **233 ferramentas**, organizadas em 24 módulos funcionais, agrupadas na Tabela 2 em dez macro-áreas.

Tabela 2: Ferramentas MCP por macro-área funcional.

Macro-área	N. de ferramentas	Módulos de origem
Esquemático (autoria, lote, hierarquia, acabamento)	65	<code>schematic.ts</code> , <code>schematic-batch.ts</code> , <code>schematic-hierarchy.ts</code> , <code>schematic-layout.ts</code>
Bibliotecas e criação de partes	31	<code>library.ts</code> , <code>library-symbol.ts</code> , <code>footprint.ts</code> , <code>symbol-creator.ts</code>
Projeto e ciclo de vida da placa	31	<code>project.ts</code> , <code>board.ts</code>
Componentes	28	<code>component.ts</code>
Exportação e fabricação	27	<code>export.ts</code>
Roteamento e autorouter	21	<code>routing.ts</code> , <code>freerouting.ts</code>
Datasheets e cadeia de suprimentos	13	<code>datasheet.ts</code> , <code>jlcpcb-api.ts</code> , <code>parts-registry.ts</code> , <code>digikkey-api.ts</code>
Regras de projeto e validação	9	<code>design-rules.ts</code> , <code>validation.ts</code>
Interface do KiCad e descoberta	6	<code>ui.ts</code> , <code>router.ts</code>
Importação de formatos legados	2	<code>eagle.ts</code> , <code>pcb-import.ts</code>
Total	233	24 módulos

Do total, 173 ferramentas (74,2 %) estão indexadas para descoberta por palavra-chave em 16 categorias, conforme a Equação 1; as 60 restantes permanecem executáveis, porém não localizáveis pelos mecanismos de busca — número que um teste automatizado congela para que só possa diminuir. Além das ferramentas, o servidor expõe 23 recursos dinâmicos com o estado do projeto (placa aberta, componentes, bibliotecas e informações do projeto) e modelos de *prompt* para tarefas recorrentes de projeto.

$$C_{desc} = \frac{173}{233} \approx 74,2\%. \quad (1)$$

4.2 Execução das suítes de teste

A Tabela 3 consolida a execução local de 10 de setembro de 2026. No total, 2.451 casos foram coletados; 2.412 foram aprovados (98,4 %), com 8 falhas e 31 casos ignorados. A suíte TypeScript passou integralmente (84 de 84).

As oito falhas concentram-se em casos que invocam o utilitário `kicad-cli` como processo externo (importação Eagle, exportação de netlist, verificação ERC e compatibilidade de formato de esquemático) e em um teste de estado de módulo legado. A análise dos rastros de execução indica sensibilidade ao ambiente de validação — instalação do KiCad por pacote *snap*, sem o

Tabela 3: Resultados da execução local das suítes de teste (10/09/2026).

Suíte	Casos	Aprov.	Falhas	Ignor.	Tempo
TypeScript (Vitest 2.1)	84	84	0	0	1,5 s
Python (pytest 9.1)	2.367	2.328	8	31	45,0 s
Total	2.451	2.412	8	31	46,5 s

diretório `/usr/share/kicad/schemas` esperado por alguns utilitários — e não a regressões funcionais nas ferramentas exercitadas pelos demais 2.328 casos aprovados. Os registros completos das execuções estão preservados no repositório (`evidencias/`).

Em integração contínua, o projeto executa a suíte TypeScript em quatro sistemas operacionais (Ubuntu 24.04, Ubuntu 22.04, Windows e macOS) com Node.js 20 e 22, a suíte Python em Ubuntu com Python 3.9 a 3.12, e dois portões de qualidade adicionais: o *lint* TypeScript e a verificação de que o inventário de ferramentas está em dia com o código.

4.3 Estudo de caso: placa seguidora de linhas de quatro camadas

O projeto analisado possui 97 *footprints* distribuídos em quatro camadas de cobre (F.Cu, In1.Cu, In2.Cu e B.Cu), espessura total de 1,6 mm e contorno de aproximadamente 82,1 mm × 80,2 mm. O esquemático é hierárquico, com nove folhas (raiz, fonte regulada, *MPPT*, pontes H, sensores ópticos reflexivos, interface USB-C e demais blocos), e a lista de materiais agrupa 37 itens. As Figura 2 e 3 mostram renderizações 3D geradas a partir do arquivo de placa; a Figura 4 apresenta a folha raiz do esquemático, cujo bloco de título registra a autoria do projeto.

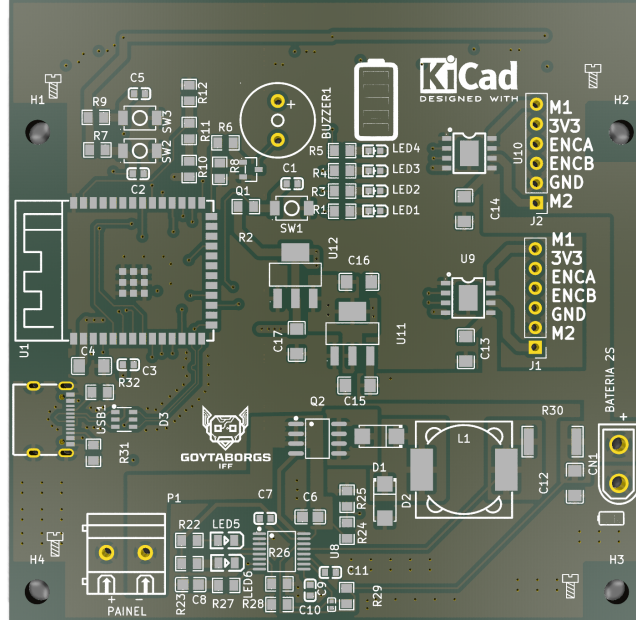


Figura 2: Vista superior da placa seguidora de linhas (renderização 3D gerada com `kicad-cli pcb render`).

A verificação de regras de projeto com `kicad-cli 9.0.7` reportou 11 violações: 9 erros e 2 avisos, detalhados na Tabela 4. Não foram encontrados itens não conectados (*unconnected items*) nem divergências de paridade entre placa e esquemático.

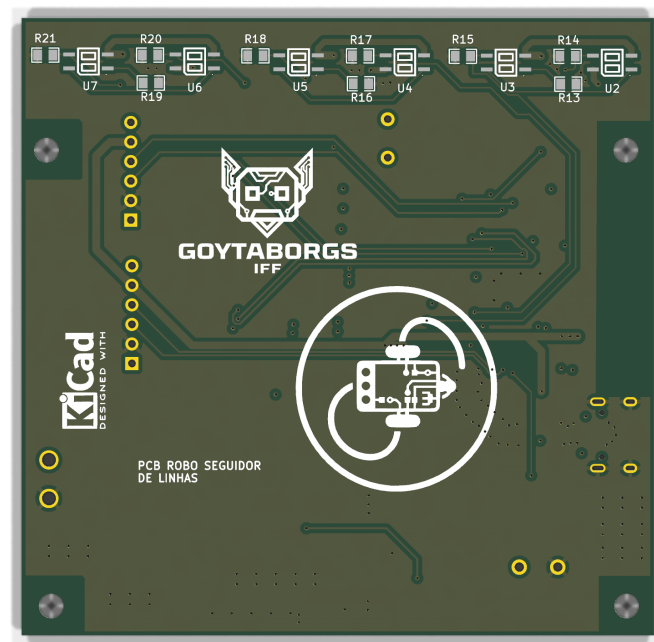


Figura 3: Vista inferior da placa, evidenciando o plano de cobre e a distribuição dos componentes SMD.

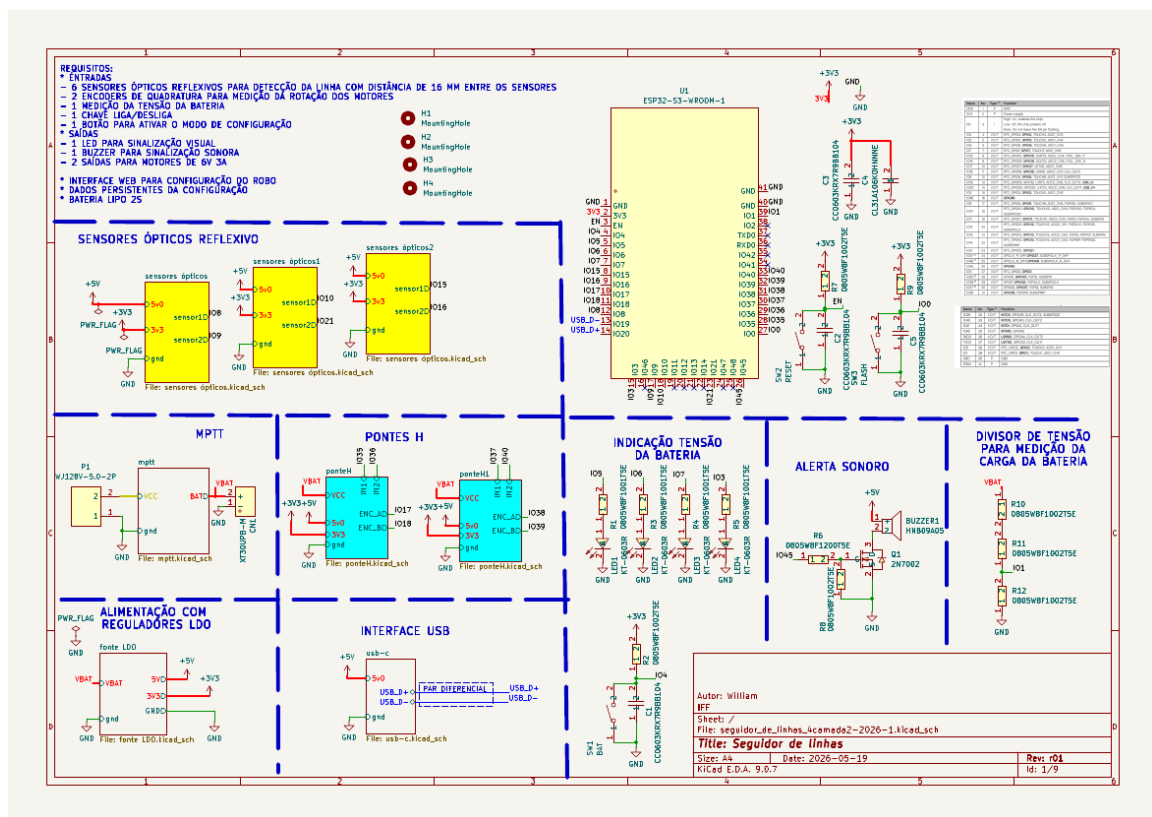


Figura 4: Folha raiz do esquemático hierárquico (nove folhas), exportada com kicad-cli sch export pdf.

Tabela 4: Violações de DRC reportadas por `kicad-cli` 9.0.7 em 10/09/2026.

Tipo	Severidade	Qtde.	Diagnóstico
<code>annular_width</code>	erro	2	Largura anular abaixo do mínimo de 0,100 mm (medição de $-0,003$ mm).
<code>clearance</code>	erro	2	Afastamento de 0,1812 mm contra 0,200 mm da classe <i>Default</i> .
<code>malformed_courtyard</code>	erro	1	<i>Footprint</i> com <i>courtyard</i> autointersectante.
<code>starved_thermal</code>	erro	4	Conexão térmica à zona incompleta (1 de 2 raios exigidos).
<code>padstack</code>	aviso	2	<i>Padstack</i> questionável (furo PTH sem cobre remanescente).

O fluxo de projeto também produziu artefatos de fabricação no diretório `production/` (lista de materiais, designadores, netlist IPC, arquivo de posições e pacote compactado da placa), demonstrando a integração entre a edição do projeto e a preparação para manufatura.

4.4 Integrações com o ecossistema de fabricação

O servidor integra três fontes de dados de componentes: a base local e a API da JLCPCB [6] (mais de 2,5 milhões de registros catalogados), o enriquecimento de *datasheets* via LCSC [10] e a API *Product Information V4* da DigiKey [2], utilizada tanto para busca pontual de componentes quanto para varredura de bibliotecas de símbolos em busca de itens obsoletos ou fora de estoque. O roteador automático Freerouting [3] é suportado via Java, Docker ou Podman. As integrações com APIs externas dependem de credenciais opcionais definidas no ambiente e degradam de forma controlada quando ausentes.

5 Discussão

5.1 Atendimento aos requisitos e à questão de pesquisa

Os resultados indicam que a questão de pesquisa — como expor o fluxo de PCB do KiCad a assistentes de IA de forma ampla, descobrível e testável — foi endereçada com sucesso nas dimensões propostas. A cobertura de R1 materializa-se nas 233 ferramentas, que abrangem desde a criação do projeto até a exportação de fabricação. R2 avançou, mas de forma parcial: 74,2 % das ferramentas estão descobríveis, e o próprio projeto reconhece as 60 restantes como dívida técnica controlada por teste automatizado. R3 e R4 refletem-se na correlação de requisições entre as camadas e na sessão fixada por *backend* — os dois mecanismos que impedem falhas silenciosas em operações destrutivas, classe de defeito recorrente em integrações desse tipo. R5 e R6 são sustentados pela matriz de integração contínua e pela execução local das suítes.

5.2 Comparação com a literatura

A literatura de interfaces agente-computador sustenta que a forma de exposição das ações afeta o desempenho do agente [22]. O servidor estudado aplica esse princípio em um domínio físico: cada operação é uma ferramenta nomeada, com esquema validado, e as operações mais frequentes formam uma lista de essenciais que ordena a busca. Diferentemente de adaptações de modelo ao domínio, como o *chip* assistente de Liu et al. [11], a solução aqui é ortogonal ao modelo: qualquer cliente compatível com o MCP 2025-06-18 obtém as mesmas capacidades, sem treinamento específico.

O servidor estudado também se distingue das automações tradicionais de KiCad — *scripts* Python isolados sobre `pcbnew` [20] — ao padronizar descoberta, tratamento de erros, recursos de

estado e integrações de fabricação em um servidor único. A verificação por `kicad-cli` [9] fecha o ciclo de qualidade fora da interface gráfica.

5.3 Limitações

- **Integração IPC não exercitada.** No ambiente de validação, o servidor IPC do KiCad estava desabilitado na configuração do usuário (`enable_server`: falso) e não havia soquete de comunicação ativo; portanto, o fluxo em tempo real não foi validado nesta execução, e os resultados referem-se ao *backend* SWIG e ao `kicad-cli`. A validação do caminho IPC é o principal trabalho futuro.
- **Cobertura de descoberta parcial.** Sessenta ferramentas não são localizáveis por busca, embora permaneçam chamáveis por nome.
- **Falhas de teste sensíveis ao ambiente.** Oito casos falharam localmente, todos dependentes de subprocessos `kicad-cli` ou de caminhos legados; a operação em CI com ambientes limpos pode comportar-se de forma distinta e deve ser monitorada.
- **Violações residuais de DRC.** A revisão analisada da placa não está pronta para fabricação sem revisão dos apontamentos: as violações de `starved_thermal`, `annular_width`, `clearance` e `padstack` indicam ajustes de geometria necessários. Este resultado deve ser lido como evidência do valor da verificação automatizada, não como defeito metodológico.
- **Avaliação restrita.** Não houve estudo com usuários nem medição de produtividade (tempo de projeto, retrabalho) contra um fluxo manual; as métricas são descritivas e derivadas de um único projeto de placa, em uma única máquina.

5.4 Ameaças à validade

Quanto à validade de construto, contagens de ferramentas e de casos de teste são proxis de capacidade e confiabilidade, não medidas diretas de utilidade para o engenheiro. Quanto à validade interna, o ambiente *snap* do KiCad influencia parte das falhas observadas; uma réplica em ambiente CI padrão é recomendável. Quanto à validade externa, a generalização para outros *softwares* de EDA exige esforço de porte; contudo, a arquitetura em duas camadas e a separação por *backends* são replicáveis.

5.5 Implicações e trabalhos futuros

Além da validação do caminho IPC, os desdobramentos naturais incluem: indexar as ferramentas restantes e ampliar a busca semântica; adicionar uma camada de políticas para operações destrutivas (confirmação, limites, auditoria); expandir a suíte para reduzir a sensibilidade a ambientes específicos; conduzir estudos com usuários medindo tempo de projeto e taxa de retrabalho; e avaliar a interoperabilidade com outros clientes MCP e versões futuras do KiCad.

6 Conclusão

Este trabalho analisou e avaliou o `mcp-kicad-vscode`, o servidor Model Context Protocol de código aberto do projeto `KiCAD-MCP-Server`, criado por *mixelpixx* [12], para automação do fluxo de projeto de placas de circuito impresso no KiCad. O servidor expõe 233 ferramentas organizadas em 24 módulos, 173 delas descobríveis em 16 categorias, além de 23 recursos dinâmicos de estado do projeto, integrações com JLCPCB, LCSC e DigiKey e suporte ao roteador automático Freerouting. A validação combinou métricas de implementação, execução das suítes de teste — 2.412 de 2.451 casos aprovados localmente, com a suíte TypeScript integralmente aprovada — e um estudo de caso, conduzido pelo autor, com uma placa seguidora de linhas de quatro camadas

(97 *footprints*, 83 redes), cuja verificação de regras reportou 11 violações residuais e zero itens não conectados.

Os resultados respondem à questão de pesquisa na medida em que demonstram ser viável, com uma arquitetura em duas camadas e disciplina de testes, expor um fluxo completo de EDA a assistentes de IA de forma padronizada e verificável. As limitações — integração IPC não exercitada localmente, cobertura de descoberta parcial e avaliação restrita a um único projeto — delimitam o alcance das conclusões e orientam a agenda de continuidade, que inclui a validação em tempo real com a interface gráfica, a ampliação da cobertura de testes e a realização de estudos com usuários.

Agradecimentos

Os agradecimentos são dirigidos a *mixelpixx*, criador do projeto **KiCAD-MCP-Server** [12], no qual este estudo se baseia, e aos contribuidores da comunidade do projeto. O estudo de caso com a placa seguidora de linhas foi conduzido pelo autor deste trabalho.

Referências

- [1] Anthropic. Introducing the Model Context Protocol. <https://www.anthropic.com/news/model-context-protocol>, 2024. Acesso em: 10 set. 2026.
- [2] DigiKey. DigiKey developer portal. <https://developer.digikey.com/>, 2026. Acesso em: 10 set. 2026.
- [3] Freerouting Project. Freerouting: Advanced PCB autorouter. <https://github.com/freerouting/freerouting>, 2026. Acesso em: 10 set. 2026.
- [4] GitHub. GitHub actions documentation. <https://docs.github.com/actions>, 2026. Acesso em: 10 set. 2026.
- [5] Paul Horowitz and Winfield Hill. *The Art of Electronics*. Cambridge University Press, Cambridge, 3 edition, 2015.
- [6] JLCPCB. JLCPCB: PCB prototype and fabrication. <https://jlcpcb.com/>, 2026. Acesso em: 10 set. 2026.
- [7] KiCad. KiCad E.D.A. documentation. <https://docs.kicad.org/>, 2025. Acesso em: 10 set. 2026.
- [8] KiCad Developers. KiCad developer documentation. <https://dev-docs.kicad.org/>, 2026. Acesso em: 10 set. 2026.
- [9] KiCad Documentation Contributors. KiCad 9.0 reference manual: Command-line interface. <https://docs.kicad.org/9.0/en/cli/cli.html>, 2025. Acesso em: 10 set. 2026.
- [10] LCSC Electronics. LCSC: Electronic components distributor. <https://www.lcsc.com/>, 2026. Acesso em: 10 set. 2026.
- [11] Mingjie Liu, Teodor-Dumitru Ene, Robert Kirby, et al. ChipNeMo: Domain-adapted LLMs for chip design. arXiv:2311.00176, 2024. <https://arxiv.org/abs/2311.00176>. Acesso em: 10 set. 2026.
- [12] mixelpixx. KiCAD MCP server: AI-assisted PCB design. <https://github.com/mixelpixx/KiCAD-MCP-Server>, 2025. Repositório upstream. Acesso em: 10 set. 2026.

- [13] Model Context Protocol. Documentation: Introduction. <https://modelcontextprotocol.io/docs>, 2025. Acesso em: 10 set. 2026.
- [14] Model Context Protocol. Specification (2025-06-18). <https://modelcontextprotocol.io/specification/2025-06-18>, 2025. Acesso em: 10 set. 2026.
- [15] OpenJS Foundation. Node.js documentation. <https://nodejs.org/docs/>, 2026. Acesso em: 10 set. 2026.
- [16] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs. arXiv:2305.15334, 2023. <https://arxiv.org/abs/2305.15334>. Acesso em: 10 set. 2026.
- [17] pytest-dev. pytest documentation. <https://docs.pytest.org/>, 2026. Acesso em: 10 set. 2026.
- [18] Python Software Foundation. Python 3 documentation. <https://docs.python.org/3/>, 2026. Acesso em: 10 set. 2026.
- [19] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. arXiv:2302.04761, 2023. <https://arxiv.org/abs/2302.04761>. Acesso em: 10 set. 2026.
- [20] SWIG Developers. Simplified wrapper and interface generator (SWIG). <https://www.swig.org/>, 2024. Acesso em: 10 set. 2026.
- [21] Vitest Team. Vitest: Next generation testing framework. <https://vitest.dev/>, 2026. Acesso em: 10 set. 2026.
- [22] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent–computer interfaces enable automated software engineering. arXiv:2405.15793, 2024. <https://arxiv.org/abs/2405.15793>. Acesso em: 10 set. 2026.
- [23] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. arXiv:2210.03629, 2023. <https://arxiv.org/abs/2210.03629>. Acesso em: 10 set. 2026.